

PARTE TEÓRICA

1. OPTIMIZACIÓN EN SISTEMAS FINANCIEROS

En una aplicación bancaria que procesa transacciones en tiempo real, lo primero que haría sería identificar los puntos críticos del flujo: base de datos, servicios externos, concurrencia, tiempos de respuesta y volumen de peticiones. En sistemas financieros no solo importa que la aplicación sea rápida, también debe ser consistente, segura y trazable.

A nivel de concurrencia, manejaría correctamente las transacciones para evitar condiciones de carrera, especialmente en operaciones sensibles como débitos, créditos o transferencias. Dependiendo del caso usaría bloqueo optimista o pesimista. Por ejemplo, si muchas operaciones pueden afectar una misma cuenta, se puede usar versionamiento con optimistic locking para evitar actualizar saldos con datos desactualizados. En escenarios más críticos, usaría bloqueo pesimista de forma controlada.

También aplicaría idempotencia en operaciones transaccionales. Esto es importante porque en una API bancaria un cliente podría reenviar una petición por timeout, y el sistema no debe duplicar una transferencia o un pago. Para eso usaría un identificador único por operación y validaría si ya fue procesada antes de ejecutarla nuevamente.

Para mejorar el rendimiento, implementaría caché en información de lectura frecuente y poco cambiante, como catálogos, tipos de cuenta, configuraciones, parámetros de negocio o estados consultados repetidamente. No cachearía saldos o datos altamente sensibles sin una estrategia clara de invalidación, porque podría mostrar información desactualizada.

En base de datos revisaría índices, planes de ejecución, consultas N+1, paginación, particionamiento si aplica y uso correcto de transacciones. También utilizaría connection pooling y separaría, cuando sea posible, operaciones de lectura y escritura. En arquitecturas más grandes aplicaría CQRS para separar modelos de consulta y comandos.

Desde el diseño, usaría una arquitectura limpia o hexagonal para separar dominio, casos de uso e infraestructura. Esto permite que la lógica crítica del negocio financiero no dependa directamente de frameworks o bases de datos. Además, usaría patrones como Strategy para reglas de negocio variables, Factory para creación controlada de objetos y Circuit Breaker para proteger la aplicación cuando un servicio externo falle.

Finalmente, complementarí todo con monitoreo, logs estructurados, métricas, trazabilidad distribuida y alertas. En sistemas financieros no basta con que el sistema funcione; también debe ser observable para detectar cuellos de botella, errores o comportamientos anómalos en producción.

2. SEGURIDAD EN APIS FINANCIERAS

Para proteger una API financiera partiría del principio de defensa en profundidad. Es decir, no dependería de una sola medida de seguridad, sino de varias capas: autenticación, autorización, validación, cifrado, auditoría y monitoreo.

Primero aseguraría la autenticación usando estándares como JWT, OAuth 2.0 u OpenID Connect, dependiendo del contexto. El token tendría expiración, claims mínimos necesarios y validación de firma. Para autorización aplicaría RBAC, separando claramente permisos de usuarios, administradores y otros roles. En Spring Boot esto lo implementaría con Spring Security, filtros JWT y reglas por endpoint.

Contra inyección SQL, evitaría construir consultas concatenando strings. Usaría JPA/Hibernate, consultas parametrizadas, Criteria API o repositorios bien definidos. También validaría entradas con Bean Validation y DTOs para no exponer directamente entidades internas.

Para CSRF, si la API es stateless y usa JWT en headers Authorization, normalmente deshabilitaría CSRF en Spring Security porque no se está usando sesión basada en cookies. Sin embargo, si se usaran cookies para autenticación, configuraría CSRF tokens, SameSite, Secure y HttpOnly.

Contra XSS, aunque el principal control está en el frontend, desde backend también evitaría devolver contenido HTML no sanitizado. En frontend escaparía valores, evitaría dangerouslySetInnerHTML y validaría los datos antes de renderizarlos. También configuraría cabeceras de seguridad como Content-Security-Policy cuando aplique.

También protegería la API con HTTPS obligatorio, cifrado de datos sensibles, hashing de contraseñas con BCrypt, control de CORS, rate limiting, bloqueo ante intentos fallidos, logs de auditoría y manejo seguro de secretos mediante variables de entorno o gestores de secretos.

Por último, manejaría errores sin exponer información interna. Por ejemplo, no devolvería stack traces ni detalles de la base de datos. Usaría respuestas HTTP claras, pero controladas, como 400 para validación, 401 para no autenticado, 403 para no autorizado y 500 para errores inesperados.

3. TRANSACCIONES EN SISTEMAS DISTRIBUIDOS

En un sistema bancario distribuido, evitaría depender directamente de una transacción global tradicional entre varios servicios, porque puede generar acoplamiento, bloqueos y problemas de disponibilidad. En su lugar, diseñaría el proceso usando consistencia eventual controlada y patrones como Saga, Outbox e idempotencia.

Por ejemplo, en una transferencia entre cuentas manejadas por diferentes servicios, dividiría el flujo en pasos: validar cuenta origen, reservar o debitar fondos, registrar la operación, acreditar en cuenta destino y confirmar la transferencia. Cada servicio tendría su propia transacción local y publicaría eventos para avanzar el proceso.

Usaría una Saga orquestada cuando el flujo requiera mayor control centralizado, por ejemplo, desde un servicio de transferencias que coordine cada paso. Si ocurre un error, ejecutaría acciones compensatorias. Por ejemplo, si se debitó una cuenta pero falló el crédito en la cuenta destino, se debe revertir o compensar el débito.

También implementaría el patrón Outbox para asegurar que los eventos se publiquen de forma confiable después de guardar los cambios en base de datos. Esto ayuda a evitar inconsistencias donde la base de datos confirma una operación, pero el evento nunca se envía al broker.

Para manejar errores usaría retries controlados, colas de mensajes, dead-letter queues, correlation IDs y logs de auditoría. Cada operación tendría un identificador único para garantizar idempotencia, de manera que si el mismo mensaje se procesa dos veces no se duplique el movimiento financiero.

En una API bancaria, además, es clave modelar estados de negocio claros: PENDING, PROCESSING, COMPLETED, FAILED o COMPENSATED. Así se puede saber en qué punto quedó una operación y permitir reprocesos seguros.

Mi enfoque sería priorizar consistencia, trazabilidad y recuperación ante fallos. En sistemas financieros, el objetivo no es solamente que una transacción sea rápida, sino que el sistema pueda explicar exactamente qué ocurrió, cuándo ocurrió y cómo se recuperó si hubo una falla.

4. PRUEBAS UNITARIAS Y DE INTEGRACIÓN

Diseñaría la suite de pruebas separando claramente pruebas unitarias, pruebas de integración y pruebas funcionales. Para una API bancaria, las pruebas deben cubrir tanto la lógica de negocio como la seguridad, persistencia y manejo de errores.

En pruebas unitarias validaría principalmente servicios de dominio y casos de uso. Por ejemplo: crear una solicitud de préstamo, validar que el monto sea mayor que cero, aprobar un préstamo pendiente, rechazar un préstamo pendiente o impedir aprobar un préstamo que ya fue aprobado. Para esto usaría JUnit 5 y Mockito, mockeando repositorios o servicios externos.

También probaría reglas de autorización a nivel de servicio cuando aplique. Por ejemplo, que un usuario normal no pueda modificar el estado de un préstamo o consultar información que no le pertenece.

En pruebas de integración usaría Spring Boot Test, MockMvc o WebTestClient, según el tipo de API. Validaría el flujo completo desde el controlador hasta la base de datos: login, creación de préstamo, consulta de préstamos, aprobación por administrador y rechazo de solicitudes inválidas. Para base de datos podría usar H2 en pruebas simples o Testcontainers si quiero acercarme más a un entorno real con PostgreSQL o MySQL.

También incluiría pruebas de seguridad con Spring Security para verificar respuestas 401 y 403, validación de JWT y acceso por roles. Esto es importante porque en una API financiera no basta con probar que el endpoint funciona; hay que probar que solo lo pueda ejecutar quien tenga permisos.

Complementaría con pruebas de errores, validaciones y casos límite: montos negativos, campos obligatorios, IDs inexistentes, conflictos de negocio, tokens inválidos y solicitudes repetidas.

Finalmente, integraría estas pruebas en un pipeline de CI/CD, por ejemplo con GitHub Actions, para que cada cambio ejecute compilación, pruebas y análisis de calidad. Esto ayuda a mantener estabilidad y confianza en el sistema a medida que crece.

5. FRONT-END: ESTADO Y AUTENTICACIÓN

En una aplicación bancaria que muestra saldos o información sensible, gestionaría el estado del frontend con mucho cuidado. No almacenaría más información de la necesaria en el navegador y evitaría guardar datos sensibles de forma permanente.

Para autenticación, usaría JWT u otro mecanismo basado en tokens. El frontend enviaría el token en el header Authorization como Bearer token. Si se usa localStorage, sería consciente del riesgo ante XSS, por lo que reforzaría validaciones, sanitización, Content Security Policy y evitaría renderizar contenido no confiable. En un entorno más sensible, preferiría cookies HttpOnly, Secure y SameSite, porque reducen la exposición del token al JavaScript del navegador.

Manejaría rutas protegidas mediante guards o protected routes. Un usuario no autenticado no debería entrar a pantallas privadas, y un usuario sin rol de administrador no debería acceder a funciones administrativas. En React lo implementaría con React Router y componentes como ProtectedRoute y RoleRoute.

Para el estado global usaría una solución controlada como Context API, Zustand, Redux Toolkit o React Query, dependiendo del tamaño del proyecto. Guardaría en estado

únicamente datos necesarios como usuario autenticado, rol y estado de sesión. Para datos bancarios como saldos o préstamos, preferiría consultarlos desde la API y refrescarlos o invalidarlos después de una operación.

También manejaría estados de carga, error y éxito para evitar que el usuario realice acciones duplicadas. Por ejemplo, deshabilitaría el botón mientras se procesa una solicitud de préstamo o una aprobación.

Para garantizar coherencia de datos, después de una operación importante volvería a consultar la información o invalidaría la caché del frontend. Por ejemplo, si un administrador aprueba un préstamo, la lista debe actualizarse y reflejar el nuevo estado.

Finalmente, aplicaría validaciones en formularios, manejo centralizado de errores HTTP, logout automático si el token expira y separación entre componentes visuales, servicios API y lógica de negocio del frontend. Esto mejora la seguridad, mantenibilidad y experiencia del usuario.

6. Spring Boot

a. ¿Qué pasa internamente cuando se usa @SpringBootApplication?

La anotación @SpringBootApplication es una anotación compuesta que agrupa principalmente tres comportamientos: @SpringBootConfiguration, @EnableAutoConfiguration y @ComponentScan.

Con @SpringBootConfiguration se indica que la clase principal contiene configuración de Spring. Con @ComponentScan, Spring busca componentes, servicios, repositorios, controladores y configuraciones dentro del paquete base y sus subpaquetes. Con @EnableAutoConfiguration, Spring Boot intenta configurar automáticamente la aplicación según las dependencias presentes en el classpath.

Por ejemplo, si Spring Boot detecta spring-boot-starter-web, configura automáticamente el servidor embebido, Spring MVC, Jackson para JSON y otros beans necesarios. Si detecta Spring Data JPA, intenta configurar EntityManager, DataSource y repositorios, siempre que existan las propiedades necesarias.

Cuando se ejecuta SpringApplication.run, se crea el ApplicationContext, se cargan las propiedades, se registran beans, se aplican auto-configuraciones, se inician componentes y finalmente se levanta el servidor embebido, como Tomcat.

En resumen, @SpringBootApplication simplifica el arranque porque evita configurar manualmente muchas piezas repetitivas. Aun así, permite personalizar el comportamiento cuando el proyecto lo necesita.

b. ¿Cómo funciona el ciclo de vida de un bean en Spring y cómo se puede intervenir?

El ciclo de vida de un bean en Spring comienza cuando el contenedor detecta su definición, ya sea por anotaciones como `@Component`, `@Service`, `@Repository`, `@Controller` o por métodos `@Bean` dentro de una clase de configuración.

Después Spring instancia el bean, inyecta sus dependencias, aplica callbacks del contenedor, ejecuta `BeanPostProcessors` y luego ejecuta métodos de inicialización si existen. Por ejemplo, se puede usar `@PostConstruct` para ejecutar lógica justo después de que el bean fue creado e inyectado correctamente.

Durante la vida de la aplicación, el bean queda disponible en el `ApplicationContext` para ser usado por otros componentes. Cuando la aplicación se apaga, Spring ejecuta métodos de destrucción como `@PreDestroy`, si están definidos.

Se puede intervenir en este ciclo de varias formas. Para lógica de inicialización simple usaría `@PostConstruct`. Para liberar recursos usaría `@PreDestroy`. Si necesito modificar o envolver beans de forma más avanzada, usaría `BeanPostProcessor`. También puedo controlar inicialización y destrucción desde métodos `@Bean` usando `initMethod` y `destroyMethod`.

En aplicaciones reales, esto es útil para inicializar datos, validar configuración, preparar clientes externos, cargar catálogos o liberar conexiones al cerrar la aplicación.

c. ¿Cómo personalizar la auto-configuración sin romper la filosofía de convención sobre configuración?

La forma correcta de personalizar Spring Boot es aprovechar sus puntos de extensión sin reemplazar innecesariamente toda la configuración automática.

Primero usaría `application.yml` o `application.properties` para modificar comportamientos estándar, como conexión a base de datos, pool de conexiones, seguridad, logging, caché o configuración del servidor. Esto mantiene la filosofía de convención sobre configuración porque se ajusta lo necesario sin reescribir todo.

Cuando necesito un comportamiento específico, puedo declarar mis propios beans. Muchas auto-configuraciones de Spring Boot usan condiciones como `@ConditionalOnMissingBean`, por lo que si defino mi propio bean, Spring Boot lo respeta y no crea el predeterminado.

También usaría perfiles como `dev`, `test` y `prod` para separar configuraciones por ambiente. Por ejemplo, H2 para pruebas locales y PostgreSQL para producción.

Si una auto-configuración no aplica, se puede excluir puntualmente con `exclude` en `@SpringBootApplication` o mediante propiedades. Sin embargo, evitaría excluir configuraciones completas sin necesidad, porque puede volver el proyecto más difícil de mantener.

Para configuraciones propias, usaría `@ConfigurationProperties`, clases `@Configuration` y beans explícitos solo donde agreguen valor. De esta manera se mantiene el equilibrio: se aprovechan las convenciones de Spring Boot, pero se personalizan los puntos que realmente necesita el negocio.